Proxy inverse et exposition d'un service en conteneur

BTS SIO 2e année SISR

Projet fil rouge · lundi 9 novembre 2026 · Blocs 2 et 3

Service web exposé sur Internet

Objectifs

- Publier en HTTPS une application qui ne sait pas le faire, avec un proxy inverse.
- Vérifier ce qui est réellement joignable depuis Internet, sans se fier à une supposition.
- Fermer un port exposé et expliquer pourquoi la première correction ne suffit pas.

Prérequis

- Votre clé SSH, utilisée en P07.
- Nginx, certificat Let's Encrypt, redirection HTTP vers HTTPS (P07).
- Port, adresse d'écoute, `ss -ltnp` (P06).

Sommaire

Étape	Travail
1	Reprendre un serveur vierge
2	Lancer l'application fournie
3	Problème 1 : HTTPS par un proxy inverse
4	Problème 2 : fermer le port 8080
5	Prouver et expliquer

Votre mission

Un éditeur vous livre une application dans un **conteneur** : un programme emballé avec tout ce dont il a besoin, qui se lance d'une commande. Elle répond en HTTP sur le port 8080. Elle ne connaît ni votre nom de domaine, ni les certificats.

Vous devez la mettre en service sur votre serveur `sisrN.tp.siovision.fr`, où **N est votre numéro**. À la fin de la séance, l'enseignant testera depuis l'extérieur :

Test	Résultat attendu
`https://sisrN.tp.siovision.fr/`	La page de l'application, avec `Name: sisrN`, sans alerte de certificat
`http://sisrN.tp.siovision.fr/`	Redirection `301` vers HTTPS
`http://sisrN.tp.siovision.fr:8080/`	Aucune réponse de l'application

Travail individuel. Votre serveur est neuf : ce que vous aviez fait en P07 n'y est plus. Notez vos preuves au fil de l'eau dans un tableau **commande / attendu / observé / conclusion** : elles serviront lundi prochain.

1. Reprendre un serveur vierge

Le serveur a été recréé : sa clé a changé, et SSH vous le signale à raison. Retirez l'ancienne empreinte, puis connectez-vous :

```
ssh-keygen -R sisrN.tp.siovision.fr
ssh etudiant@sisrN.tp.siovision.fr
sudo apt update
sudo apt install -y docker.io nginx certbot nftables
```

2. Lancer l'application fournie

```
sudo docker run -d --name app --restart unless-stopped \
  -p 8080:8080 traefik/whoami --port 8080 --name sisrN
```

Élément	Rôle
`-d` , `--name app`	En arrière-plan, sous le nom `app`
`--restart unless-stopped`	Relancée après un redémarrage du serveur
`-p 8080:8080`	Publie le port 8080 du conteneur sur le port 8080 du serveur
`traefik/whoami`	L'image de l'application
`--port 8080 --name sisrN`	Options de l'application : son port et le nom qu'elle affiche

Depuis **votre poste** (`curl.exe` sous Windows), puis dans un navigateur :

```
curl -i http://sisrN.tp.siovision.fr:8080/
```

Question 1. Le port répond-il depuis votre poste ? Si le réseau du lycée le bloque, demandez à un voisin de tester depuis **son** serveur. Pourquoi un test lancé depuis votre propre serveur ne prouverait-il rien ?

Question 2. Que montre `sudo ss -ltnp | grep 8080` ? Citez deux inconvénients à laisser cette application ainsi sur Internet.

3. Problème 1 : HTTPS par un proxy inverse

Objectif : `https://sisrN.tp.siovision.fr/` affiche l'application. Nginx reçoit les visiteurs, porte le certificat et transmet les requêtes au conteneur : c'est un **proxy inverse**.

Vous avez déjà obtenu un certificat en P07. Une différence : plus aucun fichier à servir, donc Let's Encrypt a besoin d'un dossier dédié. Créez-le, et publiez ce site sur le port 80 :

```
sudo mkdir -p /var/www/acme
sudo nano /etc/nginx/sites-available/sisrN
```

```
server {
    listen 80;
    server_name sisrN.tp.siovision.fr;

    location /.well-known/acme-challenge/ {
        root /var/www/acme;
    }
    location / {
        return 301 https://$host$request_uri;
    }
}
```

À vous, avec la fiche P07 :

1. Activez le site, validez, rechargez Nginx.
2. Obtenez le certificat avec `certbot certonly --webroot` . Quel dossier passer à `-w` ?
3. Ajoutez un bloc `server` sur le port 443. Il reprend celui de P07, sans `root` , avec :

```
    location / {
        proxy_pass http://ADRESSE:PORT;
    }
```

Question 3. Quelle adresse et quel port mettre à la place d' `ADRESSE:PORT` ? Testez.

Question 4. La page affiche ce que l'application reçoit. Que valent `Host` et `RemoteAddr` ? Pourquoi ne voit-on ni votre nom de domaine, ni votre adresse ? Ajoutez ces lignes dans la `location`, puis comparez :

```
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
```

Question 5. Vérifiez le renouvellement avec `sudo certbot renew --dry-run`. Pourquoi le port 80 garde-t-il une exception pour `/.well-known/acme-challenge/` au lieu de tout rediriger ? Que répondrait l'application à Let's Encrypt sans elle ?

Observation	Cause probable	Premier contrôle
`502 Bad Gateway`	Nginx répond, mais pas l'application	`sudo docker ps`, `curl http://127.0.0.1:8080/` sur le serveur
Page Debian au lieu de l'application	Site non activé, ou mauvais `server_name`	`ls -l /etc/nginx/sites-enabled/`
certbot : `404`	`-w` différent du `root` de la `location`	Chemin `/var/www/acme`
`Connection refused` sur 443	Bloc 443 absent ou Nginx non rechargé	`sudo nginx -t`, `sudo ss -ltnp`

4. Problème 2 : fermer le port 8080

Objectif : tout passe par Nginx. Le port 8080 ne répond plus depuis Internet.

Une première idée : un pare-feu sur le serveur
nftables filtre les paquets dans le noyau. Une erreur de règle peut vous couper de votre propre serveur, sans autre accès possible. **Programmez d'abord un filet de sécurité**, qui retirera vos règles dans cinq minutes :

```
sudo systemd-run --unit=filet --on-active=5min \
  /usr/sbin/nft delete table inet filtre
```

Remplacez le contenu de `/etc/nftables.conf` par ce fichier, en complétant la liste des ports. Ne laissez ouvert que ce dont le service a besoin, et pensez à la validation de Let's Encrypt :

```
#!/usr/sbin/nft -f
table inet filtre
delete table inet filtre
table inet filtre {
    chain entree {
        type filter hook input priority 0; policy drop;
        iif "lo" accept
        ct state established,related accept
        ct state invalid drop
        ip protocol icmp accept
        ip6 nexthdr icmpv6 accept
        tcp dport { ... } accept
    }
}
```

Avant d'appliquer, prévoyez : que répondra le port 8080 ? Le port 443 ? Notez votre prévision, puis appliquez et testez depuis l'extérieur :

```
sudo nft -c -f /etc/nftables.conf
sudo nft -f /etc/nftables.conf
```

Toujours connecté ? Annulez le filet avec `sudo systemctl stop filet.timer`. Pour tout nouvel essai de règles, relancez-le d'abord.

Question 6. Le port 8080 répond-il encore ? Votre prévision était-elle juste ? Pour vérifier que votre pare-feu fonctionne bien, lancez un petit service sur le serveur, puis testez le port 9090 depuis l'extérieur, et arrêtez-le avec `Ctrl+C` :

```
python3 -m http.server 9090
```

Comprendre ce qui se passe

Le pare-feu protège le port 9090, mais pas le port 8080. Cherchez ce que Docker a installé :

```
sudo nft list table ip nat | grep 8080
```

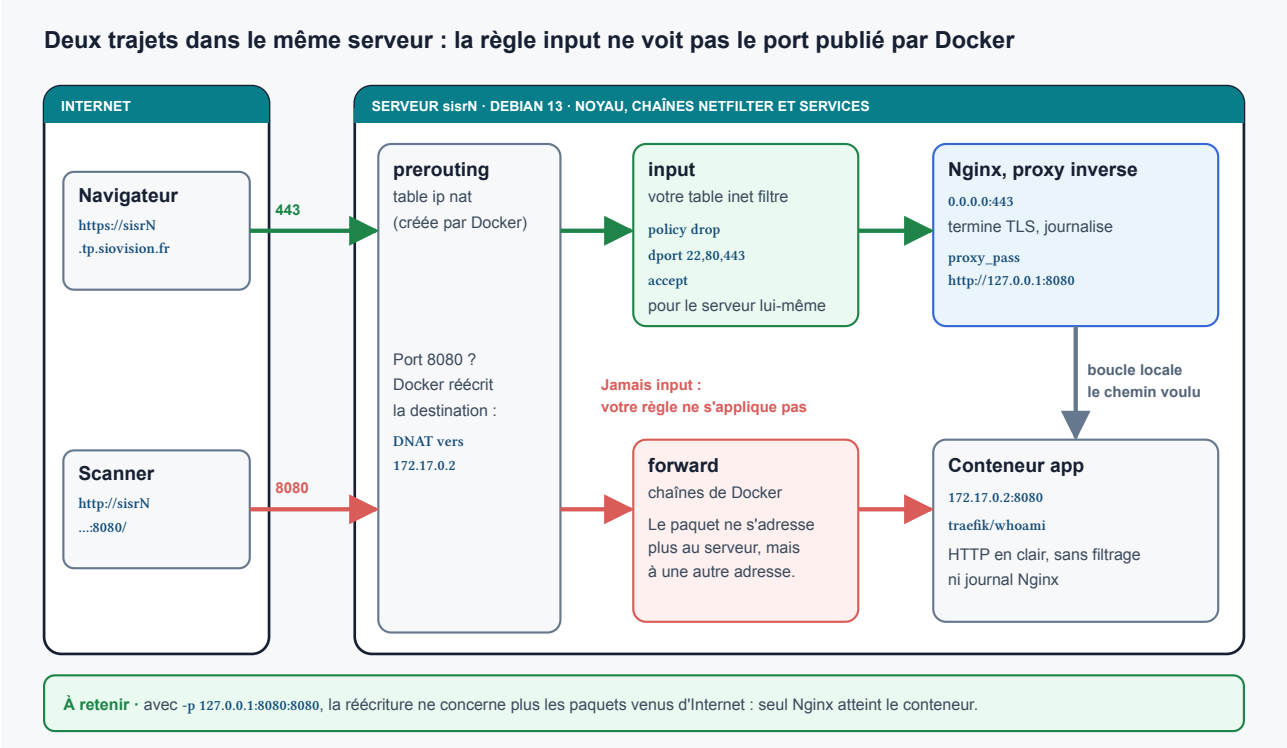


Fig. 1 - Trajet des paquets vers Nginx et vers le conteneur : le port publié par Docker est réécrit avant la chaîne input et ne la traverse jamais.

Question 7. Avec vos mots : pourquoi votre règle ne s'applique-t-elle pas au port 8080 ?

Corriger à la source

Le conteneur n'a besoin d'être joignable que par Nginx, sur le serveur même. Supprimez-le et recréez-le, en publiant son port sur **la seule boucle locale** :

```
sudo docker rm -f app
sudo docker run -d --name app --restart unless-stopped \
  -p 127.0.0.1:8080:8080 traefik/whoami --port 8080 --name sisrN
sudo ss -ltnp | grep 8080
```

Testez à nouveau HTTPS et le port 8080.

Rendre le pare-feu permanent

```
sudo systemctl enable nftables
```

Ne lancez pas `systemctl restart nftables`. À l'arrêt, ce service vide toutes les règles du serveur, y compris celles de Docker. Les conteneurs perdent alors Internet, et un port encore ouvert à tous peut sembler fermé : une fausse réussite. Pour appliquer une modification, utilisez `sudo nft -f /etc/nftables.conf`. En cas d'erreur, `sudo systemctl restart docker` recrée les règles de Docker.

Question 8. Sans nftables, le port 8080 répondrait `Connection refused` ; avec nftables, il ne répond rien. Pourquoi cette différence ? Quelles sont les deux protections en place, et que couvre chacune ?

Question 9. Complétez la matrice des flux de votre serveur :

Source	Destination	Port	Protocole	Décision	Justification
Internet	Serveur				
Internet	Serveur				
Lycée, Internet	Serveur				
Internet	Serveur	8080	TCP		
Nginx (boucle locale)	Application				

5. Prouver et expliquer

Avant la restitution, faites vérifier votre serveur par un voisin, depuis le sien :

```
curl -I http://sisrM.tp.siovision.fr/
curl -s https://sisrM.tp.siovision.fr/ | grep -E '^Name|X-Real-Ip'
curl -m 5 http://sisrM.tp.siovision.fr:8080/
```

Soyez prêt à expliquer, sans notes, le chemin d'une requête HTTPS du navigateur à l'application, et pourquoi une règle `input` ne suffit pas avec Docker.

Pour aller plus loin, dans l'ordre que vous voulez :

- Comparez `RemoteAddr` et `X-Real-Ip` dans la page avec l'adresse notée dans `/var/log/nginx/access.log`. Qui voit quoi ?
- Ajoutez un second conteneur, sur le port 8081, et publiez-le sous le chemin `/deux/`.
- Ajoutez l'en-tête `Strict-Transport-Security` avec une durée courte et expliquez son rôle.
- Redémarrez le serveur avec `sudo reboot`, puis vérifiez que tout revient sans intervention.

Repères

Un proxy inverse offre un point d'entrée unique, où se concentrent TLS, journaux et filtrage. Sur un serveur exposé, on ferme d'abord à la source (adresse d'écoute), puis on ajoute un pare-feu en seconde protection. Références : Docker, publication de ports, Docker et pare-feu, Nginx, proxy inverse, nftables, wiki.

L'IA peut expliquer un message d'erreur après une première analyse personnelle. Vérifiez toute commande avant de l'exécuter, ne transmettez jamais de clé privée ni le contenu de `/etc/letsencrypt/`, et formulez vous-même vos explications : elles seront demandées sans notes.